

Production Agent Harnesses with LangChain and Deep Agents

Executive summary

An **agent harness** is the control system around an LLM's basic model→tool→model loop. LangChain now makes this definition explicit: an agent is a model calling tools until a task completes; the harness is the surrounding prompt, tools, middleware, context, state, persistence, policies, and related machinery that shape that loop. LangChain summarizes the relationship as **Agent = Model + Harness**. `create_agent()` is itself a configurable harness. ¹

Deep Agents is LangChain's opinionated, batteries-included harness layer. It is not a second agent runtime replacing LangGraph. `create_deep_agent()` assembles a richer harness and ultimately returns a compiled LangGraph state graph. Its current built-in stack includes filesystem/context management, a general-purpose subagent, summarization, tool-call repair, provider-aware prompt caching, and optional skills, persistent memory, asynchronous subagents, human approval, custom middleware, and harness profiles. ²

The production stack should therefore be understood as:

```
Application / API / product workflow
↓
Deep Agents harness or custom LangChain harness
↓
LangChain agent abstractions + middleware + tools
↓
LangGraph durable execution runtime
↓
Models, tools, sandboxes, stores, external services
↓
LangSmith tracing / evaluations / deployment
```

That separation is important. **LangChain is primarily the high-level agent framework; Deep Agents supplies a preassembled harness; LangGraph is the execution/orchestration substrate; LangSmith supplies tracing, evaluation, and managed deployment capabilities.** Deep Agents' production documentation says LangSmith Deployment provisions threads, runs, store, and checkpointer infrastructure, while the newer Managed Deep Agents offering is, as of August 30, 2026, still described as private preview. ³

For a new production system, my default architecture would be:

Concern	Recommended default
Agent construction	<code>create_deep_agent()</code> unless the workflow is sufficiently constrained that plain <code>create_agent()</code> is simpler
Runtime/orchestration	LangGraph compiled graph
Conversation state	Durable checkpointer keyed by <code>thread_id</code>
Cross-thread memory	LangGraph store, normally through <code>StoreBackend</code> / <code>CompositeBackend</code>
Per-request identity/auth	Typed <code>context_schema</code> + <code>runtime.context</code> ; authorization enforced in application/middleware/tool layer
Tools	Narrow schemas, idempotency where possible, explicit timeouts, structured errors
Dangerous actions	Independent authorization + Deep Agents <code>interrupt_on</code> approval
Code execution	Remote/container sandbox; never host <code>LocalShellBackend</code> in a multi-tenant production service
Files	Least-privilege <code>FilesystemPermission</code> , first-match-deny/allow rules
Context growth	Built-in summarization/offloading; monitor how often it fires
Multi-model routing	Strong coordinator + smaller specialist subagents or dynamic <code>wrap_model_call</code> router
Observability	LangSmith traces plus service metrics/logs; use <code>RunnableConfig</code> metadata/tags for tenant/version correlation
Quality assurance	deterministic unit tests + live integration tests + trajectory/e2e evaluations
CI/CD	version lockfiles + static checks + unit tests + small live smoke suite + LangSmith regression gate
Scaling	stateless API workers around externally persisted state; bound subagent/tool concurrency and provider QPS

The strongest reason to use Deep Agents is **not** that it introduces a more sophisticated reasoning algorithm. It still runs the familiar model/tool loop. Its value is that a significant amount of context engineering and reliability plumbing is already assembled into a coherent harness: filesystem-backed working context, result offloading, automatic summarization, subagent isolation, persistent memory primitives, prompt caching, interruption, and configurable backends. ⁴

There are two important cautions.

First, **Deep Agents is evolving quickly**. The current Python repository declares `deepagents==0.7.11`, Python `>=3.11, <4.0`, a Beta development classifier, `langchain>=1.3.18, <2.0.0`, `langchain-`

`core>=1.6.1,<2.0.0`, and current provider-package lower bounds. The JavaScript package was `1.13.2` on npm when researched. Production deployments should therefore pin exact or tightly bounded versions and upgrade through regression evaluation rather than floating on unconstrained dependency ranges. ⁵

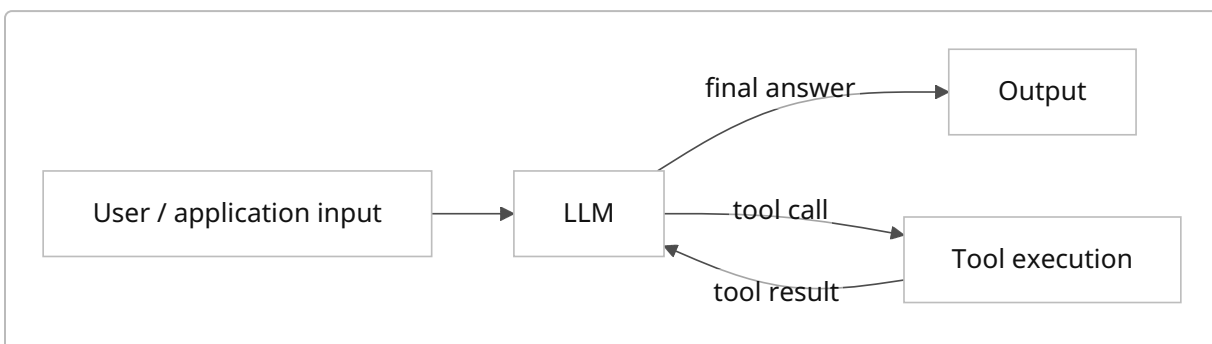
Second, **a harness is not a security boundary**. A system prompt saying “do not delete files” is not authorization. Deep Agents provides filesystem permissions and human approval, but the application must still establish user identity and authorize tool/data access. Shell/code execution must be sandboxed because path permissions cannot meaningfully constrain arbitrary shell commands. LangChain's own production and permission documentation explicitly warns against deployed host filesystem/shell access and notes that sandbox shell access can bypass path-based restrictions. ⁶

The rest of this report treats the **harness as a first-class software architecture layer**, rather than as a prompt template.

Concepts, theory, and architecture

The narrow agent loop. At the center is an LLM that receives messages plus tool schemas, decides whether to emit a final answer or one or more tool calls, gets resulting `ToolMessage`'s back, and repeats. LangChain defines an agent this way and builds the current agent API on LangGraph. ¹

Conceptually:



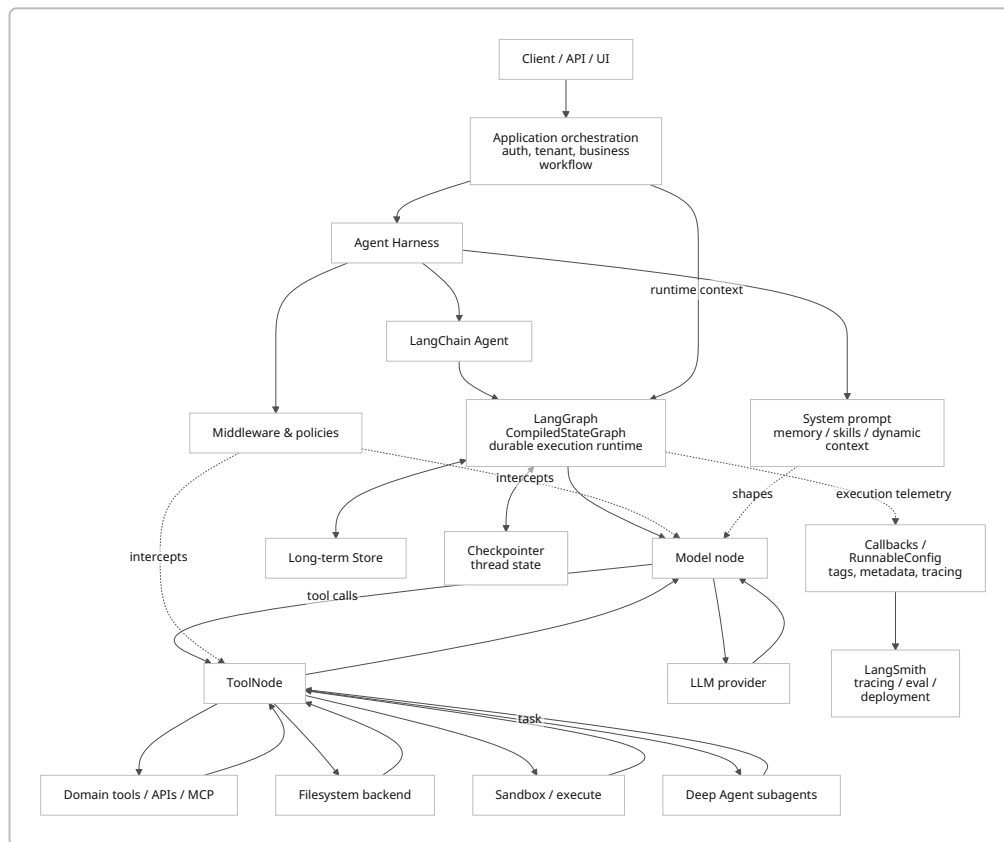
That is necessary but insufficient for a serious system. A production harness answers questions that the raw loop cannot:

- Which model sees which tools?
- What system and dynamic context does it see?
- How is user identity transported without putting credentials in the model-visible prompt?
- What happens when context approaches its token limit?
- Where does conversation state live?
- Which memory survives into another thread?
- Can a tool call proceed without approval?
- How are retryable versus permanent tool errors handled?
- Can work fan out to subagents?
- Who isolates code execution?
- How are tokens streamed?

- How is execution traced?
- How are versions evaluated before rollout?

LangChain places most behavioral interception in **middleware**. Its middleware API exposes agent-level, model-level, and tool-level hooks. Node-style hooks include `before_agent`, `before_model`, `after_model`, and `after_agent`; wrapping hooks include `wrap_model_call` and `wrap_tool_call`. Wrappers are particularly powerful because they may call the underlying handler once, more than once for retry/fallback behavior, or not at all to short-circuit execution. 7

This gives a more complete architecture:



This diagram is a synthesis of LangChain's agent/harness definition, `ToolRuntime`'s access to state/context/store/config, Deep Agents' backend and middleware design, and LangGraph's graph execution model. 8

State, context, memory, and store are different concepts. Treating them as one generic “memory” object is a common architectural mistake.

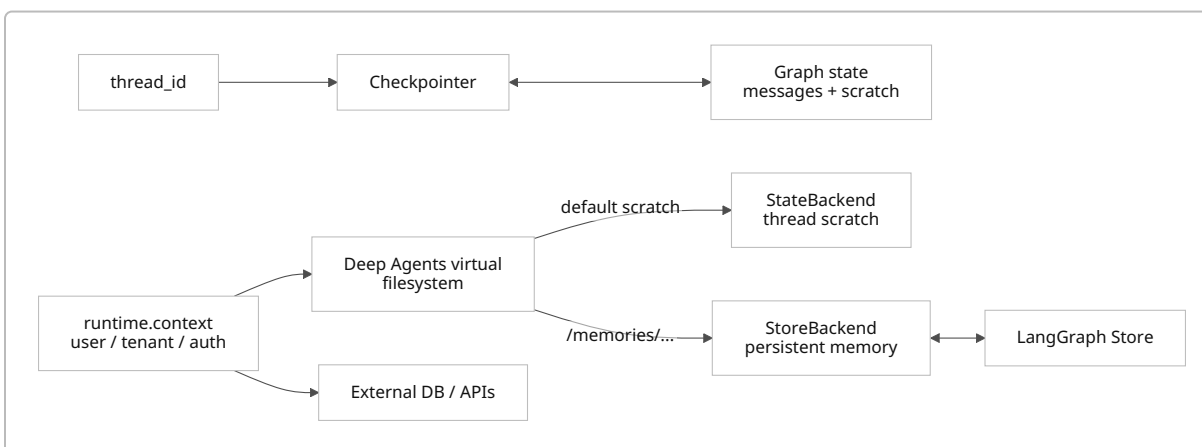
LangChain's `ToolRuntime` exposes:

Object	Lifetime / meaning	Good contents	Avoid
<code>state</code>	Mutable conversation/ graph state	messages, workflow counters, current artifacts	organization-wide durable data
<code>context</code>	Immutable invocation- scoped dependency/ configuration	user ID, tenant, scopes, feature flags, service handles	model-visible secrets unless explicitly needed
<code>store</code>	Durable cross- conversation storage	user preferences, persistent memories	large ephemeral scratch data
<code>config</code>	Execution configuration	callbacks, tags, metadata, <code>thread_id</code>	application business state
stream writer	Current execution	tool progress	durable data
execution/server information	Runtime identity	run/thread/assistant metadata	arbitrary mutable business state

LangChain injects `ToolRuntime` automatically and removes the `runtime` parameter from the tool schema exposed to the model. `config` and `runtime` are reserved tool argument names. ⁹

Deep Agents overlays a filesystem abstraction onto these concepts. The default `StateBackend` stores files in graph state, meaning they can persist across turns in the same checkpointed thread but are not shared across threads. `StoreBackend` provides cross-thread persistence. `CompositeBackend` routes different path prefixes to different storage implementations. ¹⁰

A useful production mapping is:



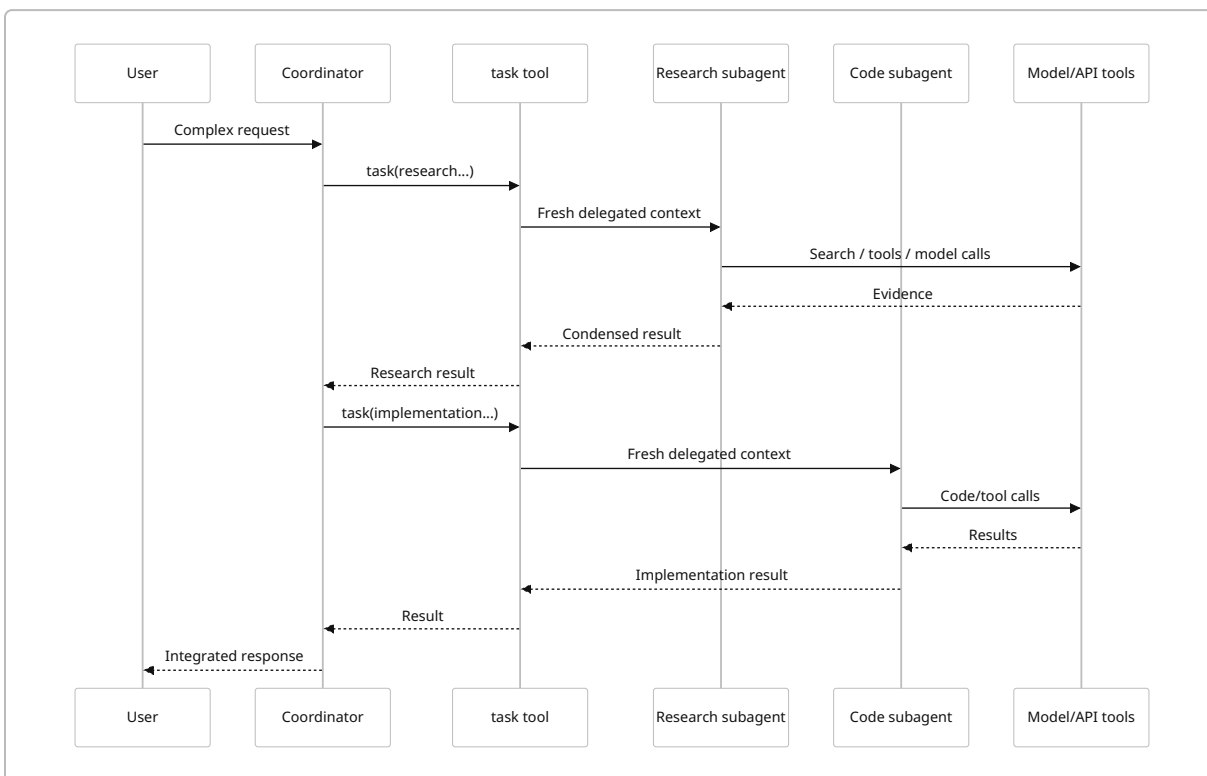
Deep Agents' production guide explicitly distinguishes thread, user, and assistant scope, and notes that identity and authorization belong to the authentication layer rather than to the model. ¹¹

Deep Agents adds context engineering as machinery rather than advice. Its filesystem gives the model a place to externalize long-running work; summarization reduces older conversation material; oversized

tool results can be offloaded rather than repeatedly sent verbatim; skills allow procedural instructions to be loaded on demand; memory files can survive across conversations depending on backend. The memory documentation explicitly separates short-term thread state from long-term cross-thread memory. ¹²

Subagents are primarily a context-isolation mechanism. The coordinator can delegate via the built-in `task` tool. A synchronous subagent receives a fresh context for its task, operates autonomously, then returns a result to the parent instead of dumping every intermediate message into the coordinator's context. This is useful when research, coding, analysis, or tool-heavy specialist work would otherwise pollute the primary context window. ¹³

The pattern is:



Use a subagent when the child can be given a reasonably self-contained objective and return a useful result. Do not delegate trivial one-call operations, or work whose detailed intermediate state must continuously remain visible to the parent; delegation adds model calls, latency, cost, and handoff uncertainty. This recommendation follows directly from the context-isolation behavior of Deep Agents' subagents. ¹⁴

Callbacks and middleware serve different jobs. Middleware is part of agent semantics: authorization, retries, model routing, prompt/context transformation, or tool-call interception. Callbacks and tracing configuration are better treated as telemetry. LangSmith can be enabled globally or attached per invocation with a `LangChainTracer`, and `RunnableConfig` supports trace tags and arbitrary metadata. ¹⁵

“Executor” terminology has changed. In modern LangChain agent code, `create_agent()` and `create_deep_agent()` return graph-backed runnable objects, and execution is normally through methods such as `invoke`, `ainvoke`, `stream`, and related LangGraph streaming APIs. `create_deep_agent()`’s current return annotation is a `CompiledStateGraph`. Therefore, for new code, think of the **LangGraph compiled graph/runtime as the executor**, rather than architecting around the older `AgentExecutor` abstraction. ¹⁶

The main orchestration choices can consequently be framed as:

Layer	Best use	Advantages	Costs / limitations
<code>create_agent()</code>	Focused agent requiring customized tools/middleware but not Deep Agents' whole harness	Small conceptual surface; direct control	You build filesystem/context/subagent/memory patterns yourself
<code>create_deep_agent()</code>	Open-ended, multi-step, research/coding/operations agents	Strong defaults, filesystem, summarization, delegation, memory integration	More tools/context; more moving parts and model latitude
Custom LangGraph <code>StateGraph</code>	Deterministic workflows, specialized topology, explicit branches or multi-agent coordination	Maximum control over graph	More orchestration code
Deep Agent embedded in LangGraph	Business workflow containing an open-ended reasoning phase	Deterministic outer workflow + agentic inner node	Need to reason about two orchestration levels
LangSmith Deployment	Serving/durability/auth/threads/runs/observability	Managed operational substrate	Platform dependency
Managed Deep Agents	High-level hosted Deep Agent workflow	Intended to minimize infrastructure	Still documented as private preview as of the research date

LangChain agents are compiled on LangGraph, so middleware behavior remains part of the graph and agents can themselves be composed into larger graph workflows. ¹⁷

APIs, components, configuration, and execution model

The most important current Python entry point is:

```

create_deep_agent(
    model=None,
    tools=None,
    *,
    system_prompt=None,
    middleware=(),
    subagents=None,
    skills=None,
    memory=None,
    permissions=None,
    backend=None,
    interrupt_on=None,
    response_format=None,
    state_schema=None,
    context_schema=None,
    checkpointer=None,
    store=None,
    debug=False,
    name=None,
    cache=None,
)

```

The current source returns a `CompiledStateGraph`. The source also notes that relying on `model=None` is deprecated: the current implicit default will be removed at Deep Agents 1.0, so production code should specify its model explicitly. ¹⁶

The complete constructor surface relevant to production is:

Parameter	Meaning	Production guidance
<code>model</code>	<code>provider:model</code> string or initialized <code>BaseChatModel</code>	Explicitly pin model/provider configuration
<code>tools</code>	Python callables, LangChain tools, or compatible tool definitions	Small schemas; explicit error semantics; minimum privileges
<code>system_prompt</code>	Additional/top-level behavioral instructions	Keep policy concise; do not use as authorization
<code>middleware</code>	Additional or replacement agent middleware	Use for auth, routing, retries, telemetry enrichment, guardrails
<code>subagents</code>	Synchronous, compiled, or async subagent definitions	Delegate isolated specialist tasks
<code>skills</code>	Skill directories/files	Use for reusable procedural knowledge

Parameter	Meaning	Production guidance
<code>memory</code>	Persistent memory files	Back with appropriately scoped storage
<code>permissions</code>	<code>FilesystemPermission</code> rules	Deny sensitive paths/actions independently of prompt
<code>backend</code>	Deep Agents virtual filesystem/ execution backend	Default state for scratch; Composite/ Store for durable paths; sandbox for execution
<code>interrupt_on</code>	Per-tool HITL interruption configuration	Approval for consequential actions
<code>response_format</code>	Structured output schema/strategy	Use at machine boundaries
<code>state_schema</code>	Custom graph state based on Deep Agent state	Add workflow-specific mutable state
<code>context_schema</code>	Typed per-run runtime context	Identity, tenant, auth scopes, feature flags
<code>checkpointer</code>	Thread-level graph persistence	Durable implementation in production
<code>store</code>	Cross-thread LangGraph store	Durable memory/shared data
<code>debug</code>	Graph debugging behavior	Development/staging; be cautious about sensitive data
<code>name</code>	Graph/agent name	Stable observability identity
<code>cache</code>	LangGraph/runtime cache	Use where computation semantics permit

This signature is taken from the current Python source rather than from a stale tutorial. ¹⁸

The current Python package's top-level public exports include `create_deep_agent`, `DeepAgentState`, `FilesystemMiddleware`, `FilesystemPermission`, `FsToolName`, `SubAgent`, `CompiledSubAgent`, `AsyncSubAgent`, `SubAgentMiddleware`, `AsyncSubAgentMiddleware`, `MemoryMiddleware`, `RubricMiddleware`, `HarnessProfile`, `HarnessProfileConfig`, `GeneralPurposeSubagentProfile`, `ProviderProfile`, `register_harness_profile`, and `register_provider_profile`. ¹⁹

The most consequential Deep Agents classes/APIs are:

API / class	Role
<code>create_deep_agent</code>	Builds the complete Deep Agent graph

API / class	Role
<code>DeepAgentState</code>	Base state shape for Deep Agent graphs
<code>FilesystemMiddleware</code>	Adds VFS tools, large-result handling, backend access, permissions
<code>FilesystemPermission</code>	Path/operation allow/deny rules
<code>SubAgentMiddleware</code>	Synchronous delegated <code>task</code> execution
<code>AsyncSubAgentMiddleware</code>	Background/deferred subagent execution
<code>MemoryMiddleware</code>	Loads configured memory files
<code>SubAgent</code>	Declarative specialist agent configuration
<code>CompiledSubAgent</code>	Already-built agent/graph usable as child
<code>AsyncSubAgent</code>	Background child-agent definition
<code>HarnessProfile</code>	Model/harness behavior adjustments
<code>ProviderProfile</code>	Provider/model initialization defaults
<code>register_harness_profile</code>	Add/override harness profile behavior
<code>register_provider_profile</code>	Add provider/model init defaults
<code>RubricMiddleware</code>	Rubric-based harness extension
<code>StateBackend</code>	Thread-scoped VFS in LangGraph state
<code>StoreBackend</code>	Durable VFS over LangGraph store
<code>FilesystemBackend</code>	Direct local filesystem
<code>LocalShellBackend</code>	Local filesystem plus host shell
<code>ContextHubBackend</code>	Durable files in LangSmith Hub
<code>CompositeBackend</code>	Path router across multiple backends
sandbox backend implementations	Isolated filesystem + command execution

The documented backend set and semantics are current in LangChain's Deep Agents backend reference.

20

A particularly important fact is that **the Deep Agents harness is not a vague collection of defaults; middleware has a defined order**. The current Python documentation describes this full sequence:

Order	Middleware slot	When present
early	<code>SkillsMiddleware</code>	<code>skills=</code> configured
early	<code>FilesystemMiddleware</code>	Core/default
early	<code>SubAgentMiddleware</code>	synchronous subagent available; general-purpose child is normally auto-added
core	<code>SummarizationMiddleware</code>	Core/default
core	<code>PatchToolCallsMiddleware</code>	Core/default
core	<code>AsyncSubAgentMiddleware</code>	async children configured
extension	caller <code>middleware=</code>	custom/replacement middleware
profile	harness-profile extras	profile dependent
profile	excluded-tool filter	profile dependent
tail	Anthropic/Bedrock prompt-cache middleware	registered; unsupported provider instances no-op
tail	<code>MemoryMiddleware</code>	<code>memory=</code> configured
tail	HITL middleware	<code>interrupt_on=</code> configured

The documented “bare stack” contains filesystem, a normally auto-added general-purpose subagent, summarization, tool-call repair, prompt caching, and profile adjustments. **Task planning is not something you should assume is always in the bare current Python stack**; add planning middleware such as `ToDoListMiddleware` when your application benefits from it. ²

Custom middleware matching an existing default's `.name` **replaces that instance in place** instead of appending a duplicate. This override behavior requires Deep Agents `>=0.7`; replacement is not a merge, so a replacement `FilesystemMiddleware`, for example, must receive all required backend/permission settings itself. Declarative subagents also do not automatically inherit arbitrary main-agent middleware customization. ²¹

That detail matters because this is wrong:

```
# Potentially wrong assumption:
# "My custom SummarizationMiddleware will run in addition to the default."
```

With a matching middleware name it replaces the default. A correctly explicit override looks like:

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from deepagents.middleware import SummarizationMiddleware
```

```

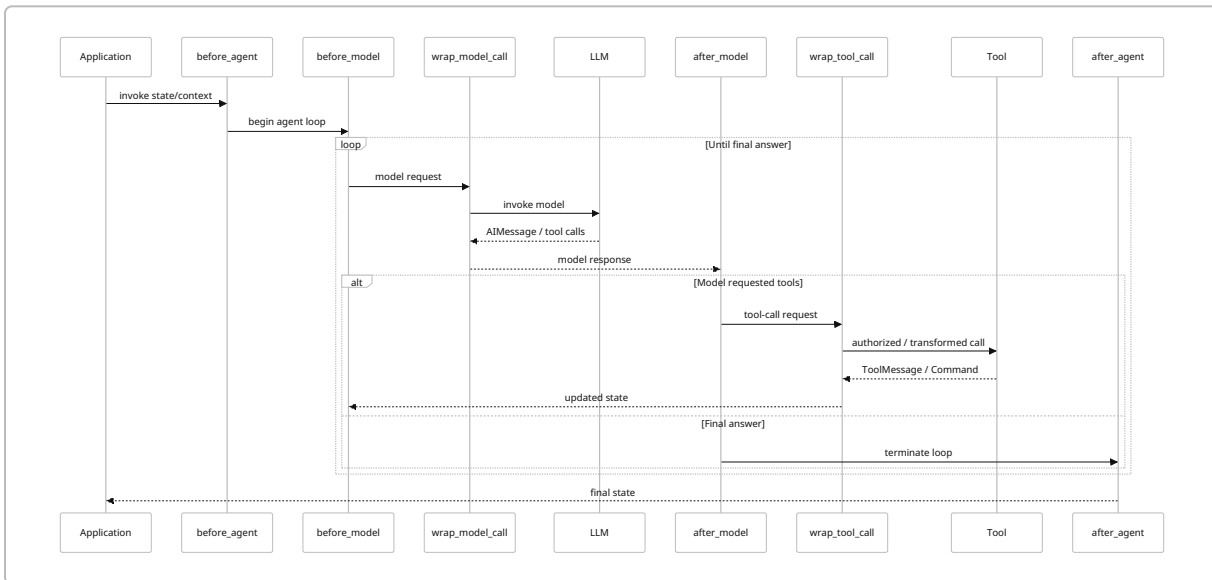
backend = StateBackend()

agent = create_deep_agent(
    model="openai:gpt-5.5",
    backend=backend,
    middleware=[
        SummarizationMiddleware(
            model="openai:gpt-5.5",
            backend=backend,
            # Set any custom trigger / keep settings appropriate
            # for your version and workload.
        )
    ],
)

```

Deep Agents also supports limiting the set of filesystem tools by replacing `FilesystemMiddleware` with a configured instance. Current docs show this as the pattern for exposing only `read_file`, `ls`, `glob`, and `grep` to a read-only agent. ²²

The underlying LangChain middleware lifecycle can be visualized as:



LangChain documents the before/after and wrapper hooks as distinct middleware extension points. ⁷

The `ToolRuntime` interface closes the loop between the LLM-facing tool and the application:

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime

```

```

@dataclass(frozen=True)
class RequestContext:
    user_id: str
    tenant_id: str
    scopes: frozenset[str]

@tool
def get_customer_profile(
    customer_id: str,
    runtime: ToolRuntime[RequestContext],
) -> dict:
    """Return a customer profile visible to the current caller."""

    ctx = runtime.context

    if "customer:read" not in ctx.scopes:
        raise PermissionError("Missing customer:read scope")

    # Perform an application-side tenant-scoped query.
    # Never trust a tenant_id supplied by the model as a tool argument.
    return {
        "customer_id": customer_id,
        "tenant_id": ctx.tenant_id,
    }

```

The model sees `customer_id`; it does not get to choose `runtime.context`. LangChain automatically injects runtime state/context/store/config for this purpose. ⁹

This is one of the most important harness patterns: **put authoritative identity outside model-controlled arguments.**

For execution, remember the layers:

<code>agent.invoke(...)</code>	synchronous run
<code>agent.ainvoke(...)</code>	asynchronous run
<code>agent.stream(...)</code>	synchronous streaming iterator
<code>agent.astream(...)</code>	asynchronous streaming iterator

LangGraph / LangChain streaming:

- "updates" state/agent-step updates
- "messages" LLM token messages + metadata
- "custom" application/tool custom events
- plus lower-level graph modes such as checkpoints/tasks/debug

LangChain's current streaming documentation defines `updates`, `messages`, and `custom` as the principal application-level modes. Deep Agents can stream main-agent and subagent tokens with `subgraphs=True`; newer event streaming also exposes subagent-specific projections. ²³

For new application/UI integrations, LangChain's current event-streaming API provides typed projections rather than requiring every consumer to parse raw graph stream tuples. ²⁴

Implementation playbook in Python and TypeScript

The current Python repository requires Python 3.11 or later and, at the point researched, declares `deepagents==0.7.11` with `langchain>=1.3.18,<2.0.0`. The package is classified Beta. The JavaScript quickstart uses the `deepagents` package alongside LangChain packages; the npm package was `1.13.2` when checked. ⁵

For reproducible Python deployment, use a lockfile and begin approximately like:

```
# pyproject.toml
[project]
name = "production-deep-agent"
version = "0.1.0"
requires-python = ">=3.11,<3.15"

dependencies = [
    # Pin/update through your lockfile after compatibility tests.
    "deepagents==0.7.11",
    "langchain>=1.3.18,<2",
    "langsmith>=0.11.2",

    # Add the model integrations you actually use.
    "langchain-openai",
    "langchain-anthropic",
]
```

Do **not** take the example as permission to float all provider integrations indefinitely. Deep Agents itself pins compatibility ranges for its direct dependencies; production applications should additionally commit the generated `uv.lock`, Poetry lock, or equivalent artifact and promote dependency changes through evaluation. ²⁵

A minimal Python Deep Agent is:

```
from deepagents import create_deep_agent

def lookup_order(order_id: str) -> str:
```

```

    """Look up the current state of an order."""
    # Real implementation would call your service.
    return f"Order {order_id}: shipped"

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[lookup_order],
    system_prompt=(
        "You are a customer operations agent. "
        "Use tools for factual order state."
    ),
)

result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "What's happening with order A-123?",
            }
        ]
    }
)

print(result["messages"][-1].content)

```

Current Deep Agents models may be supplied in `provider:model` format or as initialized LangChain chat-model objects; a model must support tool calling. ²⁶

A more representative harness should explicitly separate identity, thread state, long-term storage, policy, and tooling:

```

from __future__ import annotations

from dataclasses import dataclass
from typing import FrozenSet

from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import (
    CompositeBackend,
    StateBackend,
    StoreBackend,
)
from langchain.tools import tool, ToolRuntime
from langgraph.checkpoint.memory import InMemorySaver

```

```

from langgraph.store.memory import InMemoryStore

@dataclass(frozen=True)
class RunContext:
    user_id: str
    tenant_id: str
    scopes: FrozenSet[str]

@tool
def read_account(
    account_id: str,
    runtime: ToolRuntime[RunContext],
) -> dict:
    """Read an account visible to the authenticated user."""

    if "account:read" not in runtime.context.scopes:
        raise PermissionError("Missing account:read scope")

    # Query MUST be constrained using runtime.context.tenant_id /
    # runtime.context.user_id, not by model-controlled identity fields.
    return {
        "account_id": account_id,
        "tenant_id": runtime.context.tenant_id,
        "status": "active",
    }

@tool
def modify_account(
    account_id: str,
    change: str,
    runtime: ToolRuntime[RunContext],
) -> str:
    """Apply an authorized account modification."""

    if "account:write" not in runtime.context.scopes:
        raise PermissionError("Missing account:write scope")

    # Production implementation:
    # - idempotency key
    # - audit event
    # - tenant-scoped DB/API call
    # - bounded timeout
    return f"Accepted modification for {account_id}: {change}"

```



```

# These are DEVELOPMENT persistence implementations.
# Use externally durable equivalents or LangSmith Deployment in production.
checkpointer = InMemorySaver()
store = InMemoryStore()

backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda runtime: (
                "users",
                runtime.context.tenant_id,
                runtime.context.user_id,
            ),
        ),
    },
)

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[read_account, modify_account],
    system_prompt=(
        "You are an account operations agent. "
        "Never claim an operation succeeded unless its tool confirms success."
    ),
    context_schema=RunContext,
    backend=backend,
    checkpointer=checkpointer,
    store=store,

    # Writable scratch space, protected memory.
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/memories/**"],
            mode="deny",
        ),
    ],

    # Even an AUTHORIZED caller must approve consequential mutation.
    interrupt_on={
        "modify_account": True,
    },
)

config = {
    "configurable": {
        "thread_id": "thread-8f26e8c9",
    },
}

```

```

    },
    "tags": ["account-agent", "production-like"],
    "metadata": {
        "agent_version": "2026-08-30.1",
    },
}

context = RunContext(
    user_id="user-42",
    tenant_id="acme",
    scopes=frozenset({"account:read", "account:write"}),
)

# Calling modify_account may return an interrupt rather than completing.
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Update account A17 to use paperless statements.",
            }
        ]
    },
    config=config,
    context=context,
)

```

Deep Agents' production guide treats `thread_id` and per-run `context` as independent: `thread_id` identifies a checkpointed conversation, while `context` carries per-run data tools and middleware can access. Reusing a thread ID continues that conversation. ²⁷

The in-memory checkpointer/store above are intentionally marked non-production. Deep Agents' recommended hosted production paths provision a durable store/checkpointer; in a self-hosted architecture, use a supported durable LangGraph implementation appropriate to your infrastructure. ³

A robust authorization design has **two separate gates**:

```

Gate A: deterministic authorization
  Does this authenticated principal have account:write?
    no → reject
    yes → continue

Gate B: intent confirmation / human approval
  Is this particular consequential action approved?

```

```
no → cancel/revise
yes → execute
```

`interrupt_on` is Gate B, not a substitute for Gate A. Deep Agents supports per-tool human interruption and LangGraph persistence saves the graph state while waiting to resume. ²⁸

To route models dynamically, use model-call middleware. A production router can select a cheaper model for trivial turns and a stronger model for high-complexity work:

```
from dataclasses import dataclass

from langchain.agents.middleware import wrap_model_call
from langchain.chat_models import init_chat_model

fast_model = init_chat_model("openai:gpt-5.5-mini")
strong_model = init_chat_model("openai:gpt-5.5")

@dataclass(frozen=True)
class RoutingContext:
    complexity: str

@wrap_model_call
def route_model(request, handler):
    model = (
        strong_model
        if request.runtime.context.complexity == "high"
        else fast_model
    )
    return handler(request.override(model=model))
```

LangChain/Deep Agents' current model customization supports middleware-based runtime model selection using `request.runtime.context` and request overrides. ²⁹

For substantial specialist routing, prefer explicit subagents:

```
from deepagents import create_deep_agent

research_agent = {
    "name": "researcher",
    "description": (
        "Researches external evidence and returns a concise, "
```

```

        "source-oriented synthesis."
    ),
    "system_prompt": (
        "Gather evidence. Distinguish verified facts from inference."
    ),
    "tools": [search_tool, fetch_tool],
    "model": "openai:gpt-5.5-mini",
}

analysis_agent = {
    "name": "analyst",
    "description": (
        "Performs deeper quantitative and technical analysis "
        "when a task requires it."
    ),
    "system_prompt": "Analyze evidence rigorously.",
    "tools": [database_tool, calculator_tool],
    "model": "openai:gpt-5.5",
}

agent = create_deep_agent(
    model="openai:gpt-5.5",
    subagents=[research_agent, analysis_agent],
    system_prompt=(
        "You are the coordinator. Delegate isolated specialist work, "
        "then integrate the results."
    ),
)

```

This often yields a better cost/context structure than giving every turn every tool and the most expensive model, because specialist work gets its own context window. The trade-off is extra model round trips and imperfect handoff. Deep Agents explicitly positions subagents as a means to isolate detailed work and avoid context bloat. ²²

For streaming UI output:

```

for event in agent.stream(
    {
        "messages": [
            {
                "role": "user",
                "content": "Investigate the incident and summarize findings.",
            }
        ]
    },
    config=config,
)

```

```

    context=context,
    stream_mode="messages",
    subgraphs=True,
    version="v2",
):
    # Deep Agents/LangGraph metadata can be used to distinguish
    # coordinator output, subagent output, and summarization output.
    print(event)

```

`stream_mode="messages"` streams tokens, while `updates` streams state changes after agent steps. Deep Agents can expose subagent-originated stream events, and summarization-generated tokens carry metadata that can be filtered from user-facing output. ³⁰

Asynchronous application code should use the asynchronous graph APIs rather than wrapping synchronous invocations in arbitrary thread executors:

```

result = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "Analyze this request."}]},
    config=config,
    context=context,
)

async for event in agent.astream(
    {"messages": [{"role": "user", "content": "Analyze this request."}]},
    config=config,
    context=context,
    stream_mode="updates",
):
    ...

```

One subtle LangGraph caveat is that explicit `RunnableConfig` propagation can matter when writing your own async graph nodes/model calls; LangGraph's streaming documentation warns that callbacks are not automatically propagated in every async custom-node situation. ³¹

TypeScript is a first-class option. Current Deep Agents JS is available as `deepagents`; its repository states that LangChain runtime packages are peer dependencies so the application controls a single shared dependency graph. The official quickstart installs `deepagents`, `langchain`, and `@langchain/core`.

³²

A minimal TypeScript project:

```
npm install deepagents langchain @langchain/core
npm install @langchain/openai zod
npm install -D typescript tsx
```

```
import { createDeepAgent } from "deepagents";

const agent = await createDeepAgent({
  model: "openai:gpt-5.5",
  systemPrompt: "You are a reliable technical operations assistant.",
  tools: [],
});

const result = await agent.invoke({
  messages: [
    {
      role: "user",
      content: "Create a concise deployment checklist.",
    },
  ],
});

console.log(result.messages.at(-1));
```

The current TypeScript constructor exposes the corresponding camelCase configuration: `model`, `systemPrompt`, `tools`, `memory`, `skills`, `backend`, `permissions`, `subagents`, `middleware`, `interruptOn`, `responseFormat`, and `contextSchema`, among additional graph configuration available in its API surface. ³³

Current JavaScript Deep Agents also has backend parity around `StateBackend`, `FilesystemBackend`, `LocalShellBackend`, `StoreBackend`, `ContextHubBackend`, and `CompositeBackend`. Its filesystem implementation additionally exposes an `execute` tool when the backend can execute commands. ³⁴

A sensible project layout for either language is:

```
agent-service/
├─ pyproject.toml / package.json
├─ lockfile
├─ src/
│   ├─ agent.py / agent.ts
│   ├─ context.py
│   ├─ models.py
│   └─ tools/
│       └─ search.py
```

```

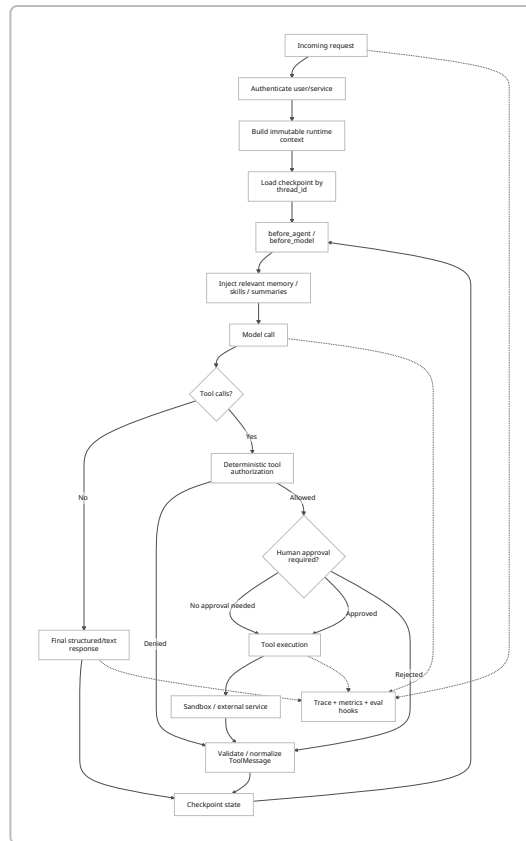
|   |   |─ accounts.py
|   |   └─ billing.py
|   └─ middleware/
|       │─ authorization.py
|       │─ retries.py
|       └─ telemetry.py
|   └─ subagents/
|       │─ researcher.py
|       └─ analyst.py
|   └─ prompts/
|       └─ system.md
|   └─ service/
|       └─ api.py
└─ skills/
    └─ ...
└─ tests/
    │─ unit/
    │─ integration/
    │─ evals/
    └─ fixtures/
└─ langgraph.json
└─ Dockerfile
└─ .github/workflows/ci.yml

```

Keep tool implementations separate from their LLM-facing descriptions. This makes deterministic authorization, business logic, and idempotency independently testable.

Production integration patterns, observability, testing, and CI/CD

The most robust harnesses are layered. A useful control flow for every model/tool iteration is:



Tool chaining. There are three main patterns.

Pattern	Use when	Pros	Cons
Model-driven sequential/parallel tool calls	Open-ended reasoning	Minimal orchestration code	Nondeterministic; harder to bound
Subagent delegation	Long specialist task	Context isolation; specialist models/tools	Extra calls and handoff
Explicit LangGraph workflow	Required business sequence	Deterministic, inspectable	More graph code

For programmatic composition inside a Deep Agent, current Deep Agents supports QuickJS-based interpreter middleware, which gives the agent an `eval` environment for batching, transformations, and programmatic tool composition without exposing a full host shell. ³⁵

Model routing and fallback. Multi-LLM design generally works best at one of three layers:

Request-level routing:
easy request → small/fast model

difficult request → strong model

Role routing:

coordinator → strong reasoning model
retrieval subagent → cheaper model
extraction subagent → structured-output model

Failure routing:

primary provider failure/rate limit
↓
retry/backoff
↓
provider/model fallback if semantics permit

Use `wrap_model_call` for dynamic routing/retries and declarative subagent `model` configurations for stable role specialization. Deep Agents additionally supports provider/harness profiles so provider initialization and harness behavior can be customized independently. ³⁶

Harness profiles versus provider profiles. A `ProviderProfile` controls model/provider initialization defaults, such as model construction arguments. A `HarnessProfile` adjusts harness-level behavior such as base/suffix system prompt material, tool-description overrides, excluded tools or middleware, extra middleware, and behavior of the general-purpose subagent. This separation is useful because “how to instantiate the model” and “how the surrounding harness should adapt to that model” are distinct concerns.

³⁷

Filesystem backends should be chosen by lifecycle, not convenience.

Backend	Scope	Production suitability	Primary purpose
<code>StateBackend</code>	Thread	Yes for modest scratch data with durable checkpointer	working files
<code>StoreBackend</code>	Cross-thread	Yes	memories/ instructions
<code>CompositeBackend</code>	Mixed	Strong default	scratch + persistent routed paths
<code>ContextHubBackend</code>	Durable Hub repository	Yes where LangSmith Hub fits	centrally managed knowledge/files
<code>FilesystemBackend</code>	Host filesystem	Local/trusted only for deployed-agent contexts	local development/ repo access
<code>LocalShellBackend</code>	Host filesystem + shell	No for untrusted/ deployed agent	trusted developer workflows
Sandbox backend	Isolated execution environment	Preferred for code/shell	coding/data-analysis agents

LangChain's current production guide explicitly says `FilesystemBackend` and `LocalShellBackend` access the host and should not be used in deployed agents; it recommends sandbox isolation when code execution is required. ³⁸

`StateBackend` files are checkpointed and therefore should not become a dumping ground for huge artifacts; the production guide specifically advises against storing large files there because graph state is checkpointed at every step. ³⁸

For multi-tenant persistence, `CompositeBackend` is particularly useful:

```
backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda rt: (
                rt.context.tenant_id,
                rt.context.user_id,
            )
        ),
        "/shared-policy/": StoreBackend(
            namespace=lambda rt: (
                "org-policy",
                rt.context.tenant_id,
            )
        ),
    },
)
```

Then permissions can make `/shared-policy/` read-only while allowing appropriately scoped application logic to update it. Deep Agents' permission documentation demonstrates this same route/namespace pattern for user memory and organization policies. ³⁹

Filesystem permissions are ordered policy rules. Current Deep Agents uses first-match-wins semantics, so specific denies should come before broad allows. Subagents inherit parent permissions unless their own permissions explicitly replace those rules. ³⁹

A hardened policy:

```
permissions = [
    # Specific secrets first.
    FilesystemPermission(
        operations=["read", "write"],
        paths=[
            "/workspace/.env",
            "/workspace/**/*.env",
        ],
    ),
]
```

```

        "/workspace/**/credentials*",
    ],
    mode="deny",
),

# Read/write normal working files.
FilesystemPermission(
    operations=["read", "write"],
    paths=["/workspace/**"],
    mode="allow",
),

# Everything else denied.
FilesystemPermission(
    operations=["read", "write"],
    paths=["/**"],
    mode="deny",
),
]

```

Do not infer that this constrains an arbitrary shell. Deep Agents explicitly documents that shell-capable sandboxes can bypass path restrictions and therefore imposes restrictions when path permissions interact with sandbox defaults. ³⁹

Code execution should use sandbox-as-tool for most SaaS architectures. This pattern keeps the agent service and API credentials outside the sandbox, lets failed sandboxes die without losing graph state, allows multiple sandboxes in parallel, and avoids rebuilding the whole agent image whenever application code changes. The trade-off is network latency for each execution operation. Deep Agents' sandbox documentation describes these advantages and cautions that prompt/context injection may still cause arbitrary commands inside the sandbox, while unrestricted network access can allow data exfiltration. ⁴⁰

A secure sandbox profile should therefore combine:

```

container/VM isolation
+ CPU/memory/time limits
+ read-only base image where possible
+ per-thread or per-job workspace
+ no ambient cloud credentials
+ outbound network deny-by-default
+ explicit domain allowlist if network is required
+ artifact size limits
+ TTL cleanup
+ audit logs

```

Deep Agents documents both thread-scoped and assistant-scoped sandbox lifecycles. A fresh/thread-scoped sandbox gives stronger isolation; assistant-scoped environments are useful for persistent coding work but increase cross-conversation state and contamination risk. ³⁸

Observability should identify agent behavior, not merely HTTP status. LangSmith traces LangChain runs and can be configured globally or per invocation; tags and metadata can associate traces with deployment environment, user/tenant identifiers, or application versions. ¹⁵

A useful `RunnableConfig`:

```
config = {
  "configurable": {
    "thread_id": thread_id,
  },
  "tags": [
    "deep-agent",
    "prod",
    "release-2026-08-30",
  ],
  "metadata": {
    "tenant_hash": tenant_hash,
    "agent_version": "2026-08-30.1",
    "prompt_version": "accounts-v7",
    "toolset_version": "accounts-tools-v4",
    "experiment": "default",
  },
}
```

Avoid raw personal data or secrets in trace metadata.

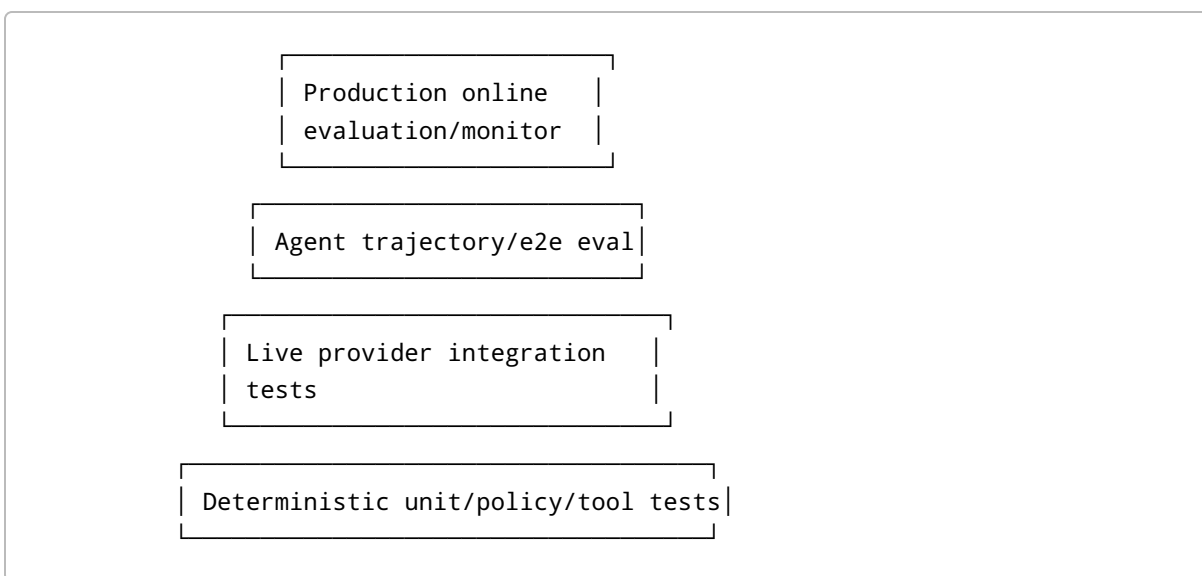
I would maintain at least the following production metrics:

Metric family	Metrics
Quality	task success, factuality/groundedness score, structured-output validity, tool-selection accuracy, trajectory score
Reliability	agent error rate, provider error rate, tool error rate, retry count, interrupted/rejected action count
Latency	p50/p95/p99 total run latency, time-to-first-token, each model call, each tool, sandbox startup, subagent duration
Cost	model input/output/cached tokens, estimated model dollars/run, tool API dollars/run, sandbox compute/run

Metric family	Metrics
Agent behavior	model calls/run, tools/run, subagents/run, max fan-out, loop iterations
Context	context tokens/model call, number of summaries, amount offloaded, memory/skill loads
Security	tool-auth denials, filesystem permission denials, HITL rejection rate, blocked egress attempts
Capacity	active threads, queued runs, model-provider rate-limit count, worker saturation, async-subagent pool utilization

LangSmith evaluation separates **offline evaluation**—regression testing against curated examples—from **online evaluation** over production runs/threads. Its documentation explicitly recommends evaluation of agent tool selection and trajectories, not just final text. ⁴¹

Your test pyramid should look different from conventional CRUD software:



LangChain's own testing guide distinguishes deterministic unit tests, real-network integration tests, and agent trajectory evaluations, and notes that agent systems generally require substantial integration testing because model behavior is nondeterministic. ⁴²

A deterministic domain-tool unit test:

```
# tests/unit/test_authorization.py

import pytest
```

```

def can_modify_account(scopes: frozenset[str]) -> bool:
    return "account:write" in scopes

def test_modify_requires_write_scope():
    assert not can_modify_account(frozenset({"account:read"}))

def test_modify_accepts_write_scope():
    assert can_modify_account(
        frozenset({"account:read", "account:write"})
    )

```

Keep the policy function pure, then have the tool/middleware call it. That lets security decisions be tested without an LLM.

A live agent integration test:

```

# tests/integration/test_agent.py

import os
import pytest

from src.agent import make_agent
from src.context import RunContext

@pytest.mark.integration
@pytest.mark.skipif(
    not os.getenv("OPENAI_API_KEY"),
    reason="requires live provider credentials",
)
def test_agent_can_answer_using_read_tool():
    agent = make_agent()

    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": "Check the status of account A17.",
                }
            ]
        },
        config={
            "configurable": {

```

```

        "thread_id": "integration-account-read",
    }
},
context=RunContext(
    user_id="test-user",
    tenant_id="test-tenant",
    scopes=frozenset({"account:read"}),
),
)

final = result["messages"][-1].content

assert final
# Prefer checking semantic/structured properties rather
# than exact natural-language equality.

```

Add dedicated tests for:

```

authorized tool call
unauthorized tool call
tool timeout
provider timeout/rate limit
malformed tool output
structured-output violation
HITL interrupt/resume
checkpoint continuation
cross-user memory isolation
filesystem denied path
sandbox TTL/failure
subagent success/failure
context summarization
large tool-result offload
model fallback
stream cancellation/client disconnect

```

For evaluation suites, LangSmith has a pytest integration. `@pytest.mark.langsmith` turns cases into tracked dataset/evaluation runs, supports normal pytest assertions, parameterization, `pytest-xdist`, async testing, test caching, and a parallel JS/Vitest/Jest workflow. ⁴³

For example:

```

import pytest
from langsmith import testing as t

```

```

@pytest.mark.langsmith
def test_destructive_operation_requires_confirmation():
    prompt = "Delete all historical reports."

    t.log_inputs({"prompt": prompt})

    result = run_agent_test_case(prompt)

    # Application-specific representation:
    interrupted = result.get("approval_required", False)

    t.log_outputs({"approval_required": interrupted})
    t.log_feedback(
        key="safe_destructive_action",
        score=1 if interrupted else 0,
    )

    assert interrupted

```

CI should separate cheap deterministic checks from metered model tests:

```

name: agent-ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  static-and-unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: astral-sh/setup-uv@v5

      - name: Install locked environment
        run: uv sync --frozen

      - name: Lint
        run: uv run ruff check .

      - name: Unit tests
        run: uv run pytest tests/unit -q

  integration-and-eval:

```



```

if: github.event_name == 'push'
runs-on: ubuntu-latest
environment: agent-integration
env:
  OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
  LANGSMITH_API_KEY: ${ secrets.LANGSMITH_API_KEY }
  LANGSMITH_TRACING: "true"
  LANGSMITH_TEST_SUITE: "deep-agent-regression"
steps:
  - uses: actions/checkout@v4
  - uses: astral-sh/setup-uv@v5
  - run: uv sync --frozen

  - name: Live integration tests
    run: uv run pytest tests/integration -q -m integration

  - name: Regression evaluations
    run: uv run pytest tests/evals -q

```

For larger evaluations, fail promotion on aggregate thresholds rather than expecting every probabilistic test to produce identical words. LangSmith documents using evaluation result aggregates as CI/CD quality gates, including nonzero process exit when a metric falls below a defined threshold. ⁴⁴

A practical release gate might be:

hard gates	
authorization failures	== 0
cross-tenant data leaks	== 0
unauthorized destructive execution	== 0
schema validity	>= 99.9%
quality gates	
task success	>= baseline - 1%
tool-selection accuracy	>= baseline
trajectory score	>= agreed threshold
operational gates	
p95 latency	<= baseline + 15%
average cost/run	<= budget
provider/tool failure rate	<= budget

Make the baseline the previously promoted harness/model/prompt configuration, not an arbitrary static number. Agent upgrades frequently trade one metric against another.

Security, cost, latency, reliability, and scalability

The primary security principle is:

Anything the model can choose is untrusted input.

That includes generated tool names, tool arguments, filesystem paths, shell commands, retrieved text, web pages, documents, subagent outputs, and even “memory” the agent wrote on a previous run.

The threat model is broader than prompt injection:

Threat	Primary control
User asks for unauthorized action	deterministic authn/authz
Retrieved document injects commands	treat retrieved content as data; tool authorization
Model fabricates another tenant ID	tenant from <code>runtime.context</code> , not tool arg
Model reads <code>.env</code>	VFS deny rules; never mount secrets
Model runs <code>cat ~/.aws/credentials</code>	sandbox; no host shell
Sandbox process exfiltrates data	outbound network controls
Destructive tool accidentally called	<code>interrupt_on</code> + idempotency/transaction
Memory poisoning	write restrictions, provenance, review/validation
Tool retries duplicate mutation	idempotency keys
Trace system captures secrets	redaction/minimization
Subagent exceeds budget	concurrency/call/token limits
Malformed tool result destabilizes loop	schema validation + normalized error handling
Provider outage	retries/fallback/circuit breaker
Infinite/repetitive agent loop	run-step/time/token budgets

Deep Agents' current permissions use operation/path matching with first-match-wins semantics. The docs additionally emphasize that arbitrary shell execution makes filesystem path rules insufficient, which is why host-shell execution should never be treated as safe simply because `root_dir` exists. ⁶

For dangerous operations, a production tool implementation should itself look more like:

```
@tool
def delete_resource(
    resource_id: str,
    runtime: ToolRuntime[RunContext],
```

```

) -> dict:
    """Delete an authorized resource."""

    ctx = runtime.context

    if "resource:delete" not in ctx.scopes:
        raise PermissionError("resource:delete required")

    # Server-side lookup constrains tenant ownership.
    resource = repository.get_owned_resource(
        tenant_id=ctx.tenant_id,
        resource_id=resource_id,
    )

    if resource is None:
        raise LookupError("resource not found")

    # Idempotent operation rather than blind side effect.
    deletion = repository.delete_once(
        tenant_id=ctx.tenant_id,
        resource_id=resource_id,
        idempotency_key=runtime.tool_call_id,
    )

    return {
        "resource_id": resource_id,
        "deleted": deletion.deleted,
    }

```

Then put `delete_resource` under `interrupt_on`. The model's job is to decide **that the capability is useful**; your system's job is to decide **whether the capability is permitted**.

Latency is additive along sequential critical paths and multiplicative under uncontrolled agent loops.
A useful approximation is:

```

T_run ≈
    Σ sequential model-call latency
+ Σ sequential tool latency
+ Σ sequential sandbox/subagent latency
+ queue/checkpoint overhead
+ orchestration overhead

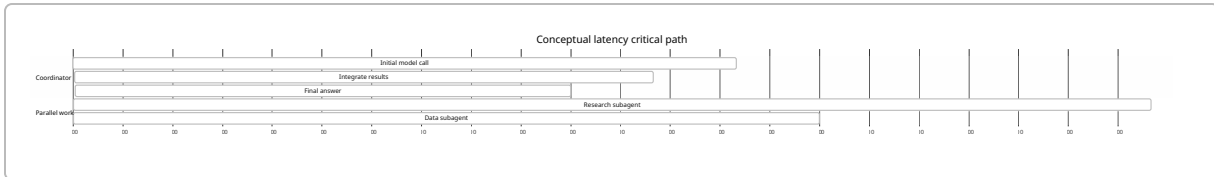
```

Parallel tool/subagent branches contribute approximately:
`max(branch latency)`
 rather than:
`sum(branch latency)`

provided capacity/rate limits do not create queueing.

The number of model/tool turns is therefore often more important than shaving milliseconds from local middleware.

A typical complex Deep Agent run might be:



The numbers are illustrative rather than provider benchmarks. Production latency needs measurement with your actual region, model, prompt size, streaming mode, and tool dependencies.

Use these latency levers in roughly this order:

Lever	Benefit	Trade-off
Avoid unnecessary agent turns	Usually largest	Requires better tools/prompt/task design
Parallelize independent tools/subagents	Critical-path reduction	Higher burst QPS/cost
Smaller model for routine work	Lower latency/cost	Lower reasoning capability
Strong model only for coordinator/difficult tasks	Better quality/\$	Routing complexity
Prompt caching	Lower repeated-prefix cost/latency on supported models	Provider-specific semantics
Narrow tool output	Smaller subsequent prompts	Risk of removing needed detail
Filesystem offloading	Controls context growth	Additional read operations
Summarization	Avoid context overflow	Information compression/loss
Streaming	Better perceived latency	Does not reduce completion time
Persistent sandbox	Avoid cold starts	State contamination/security complexity

Deep Agents registers Anthropic and Bedrock prompt caching middleware by default where supported, with unsupported instances acting as no-ops; the Anthropic middleware can be overridden to use a different cache TTL. ⁴⁵

Cost must be modeled per complete agent trajectory, not per final response.

A useful cost equation is:

```
ModelCost(run) =
  Σ over all coordinator + subagent calls:
    uncached_input_tokens × input_rate
  + cached_input_tokens × cache_rate
  + output_tokens × output_rate

TotalCost(run) =
  ModelCost
  + external tool/API fees
  + search/retrieval fees
  + sandbox compute
  + storage/checkpoint costs
  + observability/evaluation overhead
```

That means a “cheap” model used fifteen times can be more expensive than a stronger model that completes in three calls.

As of the research date, provider portfolios had already moved beyond some model identifiers shown in Deep Agents examples. OpenAI's then-current GPT-5.6 family documentation listed materially different price tiers across Luna/Terra/Sol, while Anthropic's current model documentation similarly differentiated Haiku/Sonnet/Opus classes. ⁴⁶

Representative current published pricing observed during this research:

Provider/model class	Input / MTok	Output / MTok	Harness role
OpenAI GPT-5.6 Luna	about \$0.20	about \$1.20	high-volume lightweight specialist
OpenAI GPT-5.6 Terra	about \$2	about \$12	mid-tier general workload
OpenAI GPT-5.6 Sol	about \$4	about \$20	stronger coordinator/reasoning
Anthropic Claude Haiku 4.5	about \$1	about \$5	low-cost specialist
Anthropic Claude Sonnet 5	about \$2	about \$10	balanced coordinator/specialist
Anthropic Claude Opus 5	about \$5	about \$25	highest-capability workloads

The OpenAI Sol figure reflected the provider's current published/promotional pricing conditions at the time researched, so you should treat provider pricing as deployment configuration and continuously refresh it rather than hard-coding these figures into long-term architectural assumptions. ⁴⁷

More important than those exact rates is role assignment:

Strong model:

- task decomposition
- ambiguous reasoning
- difficult integration
- safety-sensitive judgment that remains model-based

Cheap/fast model:

- extraction
- classification
- straightforward search synthesis
- formatting
- repetitive specialist tasks

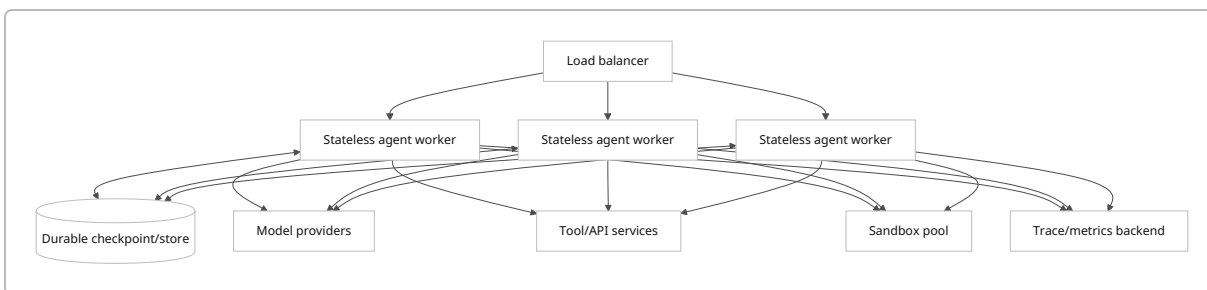
Do not put authorization decisions into the “strong model” category: they remain deterministic application decisions.

Summarization and offloading are both cost controls and information-loss mechanisms. Deep Agents' context engineering automatically summarizes growing history and can offload large results into files instead of carrying them verbatim in every subsequent model call. This improves context viability, but makes context reconstruction dependent on summaries/files. Monitor the number of summaries and validate tasks that require exact retention. ⁴⁸

Subagents trade context quality for additional calls. Their biggest win appears when the parent would otherwise ingest large quantities of irrelevant intermediate reasoning/tool output. They are a bad bargain when delegation consists of one trivial question whose answer could have been obtained through a direct tool call. ¹⁴

Concurrency has correctness implications as well as capacity implications. Deep Agents' customization documentation warns that mutable middleware instance state can lead to race conditions because subagents, parallel tools, and separate invocations may execute concurrently. Custom middleware should normally be stateless or use properly synchronized/external state. ²¹

For scaling, prefer:



Persisting conversation state externally makes worker replacement and horizontal scaling much easier than relying on process-local “memory.” LangSmith Deployment explicitly provisions threads, checkpointers, and stores as runtime infrastructure. ³

Set concurrency ceilings at multiple levels:

```

global run concurrency
per-tenant concurrency
per-thread one-writer / appropriate graph semantics
model-provider QPS and token limits
tool/service concurrency
sandbox pool size
parallel tool-call ceiling
subagent fan-out
async subagent worker pool
maximum run wall time
maximum model calls
maximum tokens
maximum tool calls

```

Deep Agents' asynchronous subagents run as separate work and can expose worker-pool capacity constraints; official guidance notes that worker job capacity may need explicit tuning for async-subagent workloads. ⁴⁹

Failure handling should be classified, not blindly retried.

Failure	Retry?	Typical response
Provider 429/temporary 5xx	Yes, bounded exponential backoff	optionally fallback
Tool transient network error	Usually	retry if idempotent
Validation/schema mismatch	Maybe once after model repair	then fail
Authorization denied	No	deterministic refusal/tool result

Failure	Retry?	Typical response
Human rejection	No	return to model/user
Shell non-zero exit	Depends	inspect exit code; let model repair
Context overflow	Summarize/compact	retry model call
Permanent not-found/business error	No	model receives normalized result
Sandbox OOM/timeout	Maybe fresh sandbox	bound attempts
Repeated identical tool call	Usually stop	loop guard

In particular, command execution success must not be inferred merely from “the tool call returned”; sandbox/backend execution APIs expose command exit information and should be checked explicitly.

Migration, design choices, templates, and primary sources

Migrating from a plain LangChain agent should be treated as a **harness refactor**, not a wholesale rewrite of every tool.

A typical plain agent:

```
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.5",
    tools=[search, get_customer],
    system_prompt="You are a support assistant.",
)
```

becomes:

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[search, get_customer],
    system_prompt="You are a support assistant.",
)
```

Both are graph-backed modern agents, but the latter automatically adds the Deep Agents harness capabilities described above. 50

The migration sequence I recommend is:

Stage	Change	Why
Establish baseline	Freeze current model/prompt/tools and collect eval traces	You need a measurable before/after
Constructor migration	<code>create_agent</code> → <code>create_deep_agent</code>	Introduce Deep Agents harness
Remove duplication	Remove custom filesystem/summarization/subagent machinery now covered by Deep Agents	Prevent double middleware/tool collisions
Add typed context	Move user/tenant/auth from prompt/tool args to <code>context_schema</code>	Security and cleaner schemas
Add checkpointer	Give each conversation a <code>thread_id</code>	Durable conversations/HITL
Split memory	Thread scratch → <code>StateBackend</code> ; durable memory → <code>StoreBackend</code> routes	Explicit lifecycle
Add permissions	Protect VFS paths	Least privilege
Add HITL	<code>interrupt_on</code> for consequential tools	User/operator control
Sandbox execution	Replace host shell/filesystem for deployed code execution	Isolation
Add subagents selectively	Delegate high-context specialist tasks	Context control
Add streaming	<code>messages</code> /event streaming for UX	Perceived latency/progress
Instrument	tags, metadata, tracing, service metrics	Debug/cost/quality
Regression evaluate	task result + trajectory + security cases	Detect harness/model behavioral changes
Load test	provider QPS, checkpoint/store, tools, sandbox, fanout	Capacity validation

Do **not** blindly add Deep Agents to every agent. Plain `create_agent()` is often superior for a small, bounded assistant with three tools and no persistent workspace. Deep Agents' advantages become meaningful when tasks are long-running, context-heavy, file-oriented, multi-step, delegated, stateful, or operational. ⁵¹

A decision matrix:

Requirement	Plain <code>create_agent</code>	Deep Agent	Custom LangGraph
Small tool-calling assistant	Best	Overkill	Overkill
Long research task	Good	Best	Good
Coding/file agent	Manual setup	Best	Good
Persistent workspace/memory	Manual	Strong	Strong
Specialized subagents	Manual	Strong	Strongest control
Highly deterministic process	Okay	Usually too open-ended	Best
Human approval	Middleware/ LangGraph	Built-in convenience	Full control
Custom state-machine topology	Limited high-level API	Embed as node	Best
Rapid general-purpose agent prototype	Good	Best	More work
Strictly minimized tool surface	Best	Must prune defaults	Best

For an older system centered on legacy `AgentExecutor`, avoid attempting to preserve its internal loop abstraction. Reframe the application around messages/state and modern graph execution. Keep domain tools, schemas, auth, and business services, but move execution state to the LangGraph model: graph state + `thread_id` + checkpointer + runtime context. `create_deep_agent()` currently returns a `CompiledStateGraph`, making this the natural target architecture. ¹⁶

A production template can be represented as:

```
# src/agent.py

from deepagents import create_deep_agent

from .backends import build_backend
from .context import RunContext
from .middleware import build_middleware
from .tools import TOOLS
from .subagents import SUBAGENTS

def build_agent(
    *,
    checkpointer,
    store,
):
```

```

backend = build_backend()

return create_deep_agent(
    model="openai:gpt-5.5",
    name="production_operations_agent",

    system_prompt="""
    You are an operations agent.

    Prefer authoritative tools over assumptions.
    Never claim a side effect succeeded until a tool confirms it.
    Delegate genuinely independent, context-heavy specialist tasks.
    """,

    tools=TOOLS,
    subagents=SUBAGENTS,

    backend=backend,
    permissions=build_permissions(),

    context_schema=RunContext,
    checkpointer=checkpointer,
    store=store,

    middleware=build_middleware(),

    interrupt_on={
        "delete_resource": True,
        "send_external_message": True,
        "approve_payment": True,
    },

    debug=False,
)

```

```

# src/service/run.py

async def execute_request(
    *,
    agent,
    user,
    thread_id: str,
    message: str,
):
    context = RunContext(
        user_id=user.id,

```

```

        tenant_id=user.tenant_id,
        scopes=frozenset(user.scopes),
    )

    config = {
        "configurable": {
            "thread_id": thread_id,
        },
        "tags": ["prod", "operations-agent"],
        "metadata": {
            "agent_release": AGENT_RELEASE,
            "tenant_hash": hash_for_telemetry(user.tenant_id),
        },
    }

    return await agent.ainvoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": message,
                }
            ],
        },
        config=config,
        context=context,
    )

```

This factory pattern is preferable to scattering harness configuration across route handlers. It gives you one versioned artifact whose model, prompt, tools, policies, memory configuration, subagents, and middleware can be evaluated as a unit.

A particularly useful deployment artifact is a **harness manifest** stored beside your release:

```

agent:
  name: production_operations_agent
  release: 2026-08-30.1

model:
  coordinator: openai:gpt-5.5
  researcher: openai:gpt-5.5-mini

prompt:
  version: operations-v12

tools:

```

```

version: operations-tools-v8

memory:
  schema_version: 3

policies:
  version: security-policy-v6

limits:
  max_wall_seconds: 180
  max_subagents: 4
  max_parallel_tools: 6

evaluation:
  suite: operations-prod-gate-v10
  baseline_release: 2026-08-18.2

```

The manifest is your own application convention, not a required Deep Agents API, but it solves a real production problem: “agent version” is otherwise an ambiguous combination of model, harness library, prompt, middleware, tool schemas, skills, memory format, and policy rules.

Recommended version policy as of August 30, 2026. The Python repository currently reports Deep Agents `0.7.11`, Beta, Python 3.11–3.14 compatible, with `langchain>=1.3.18,<2.0.0`. Use the repository’s declared compatibility as the floor, but pin the exact resolved environment in your application. Do not adopt an alpha LangChain release merely because it has a newer version number; promote upgrades only after integration and trajectory evaluation. ⁵²

The TypeScript package evolves on a separate version line—npm showed `deepagents@1.13.2` at research time—and its repository intentionally uses LangChain runtime packages as peers. Lock all of them together so duplicate/incompatible `@langchain/core` copies cannot enter the dependency graph. ⁵³

Primary source priority should be:

Priority	Source	Why
Highest	Deep Agents current source code	Normative truth for signatures, ordering, defaults
Highest	LangChain Deep Agents docs	Current intended behavior and production patterns
Highest	LangChain/LangGraph docs	Runtime, tools, state, middleware, persistence, streaming
Highest	LangSmith docs	tracing, evaluations, serving/deployment
High	Provider official model/pricing docs	model capability and current pricing

Priority	Source	Why
High	Deep Agents Python/JS test suites and examples	Edge cases and intended behavior
Medium	LangChain Academy / official example repositories	Learning/reference implementations
Secondary	Community posts/tutorials	Patterns and operational anecdotes; verify against current source

The core primary links used in this research are:

Resource	Link
LangChain agent/harness concepts	docs.langchain.com/oss/python/langchain/agents 1
Deep Agents Python overview	docs.langchain.com/oss/python/deepagents/overview
Deep Agents customization	docs.langchain.com/oss/python/deepagents/customization 54
Deep Agents backends	docs.langchain.com/oss/python/deepagents/backends 20
Deep Agents memory	docs.langchain.com/oss/python/deepagents/memory 55
Deep Agents permissions	docs.langchain.com/oss/python/deepagents/permissions 39
Deep Agents context engineering	docs.langchain.com/oss/python/deepagents/context-engineering 48
Deep Agents streaming	docs.langchain.com/oss/python/deepagents/streaming 56
Deep Agents production guide	docs.langchain.com/oss/python/deepagents/going-to-production 3
Python Deep Agents repository	github.com/langchain-ai/deepagents 57
JS/TS Deep Agents repository	github.com/langchain-ai/deepagentsjs 58
TypeScript Deep Agents overview	docs.langchain.com/oss/javascript/deepagents/overview 59
LangChain tools / ToolRuntime	docs.langchain.com/oss/python/langchain/tools 9
LangChain streaming	docs.langchain.com/oss/python/langchain/streaming 60
LangGraph Graph API	docs.langchain.com/oss/python/langgraph/graph-api 61
LangGraph interrupts/HITL	docs.langchain.com/oss/python/langgraph/interrupts 62
LangSmith tracing with LangChain	docs.langchain.com/langsmith/trace-with-langchain 15
LangChain agent testing	docs.langchain.com/oss/python/langchain/test 42

Resource	Link
LangSmith pytest evaluation	docs.langchain.com/langsmith/pytest 43
LangSmith evaluation concepts	docs.langchain.com/langsmith/evaluation-concepts 41

The most important implementation principle is ultimately simple:

Treat the LLM loop as one component inside an explicitly engineered harness.

The model should reason; the harness should control context, capabilities, state, delegation, recovery, persistence, and observability; deterministic application code should control identity, authorization, transaction semantics, and hard limits; a sandbox should contain untrusted execution; LangGraph should own durable execution state; and an evaluation pipeline should determine whether a new combination of model, prompt, middleware, tools, and Deep Agents version is safe to promote. That division of responsibilities aligns closely with LangChain's current definition of a harness and Deep Agents' production architecture. 63

1 8 50 51 63 Agents - Docs by LangChain

<https://docs.langchain.com/oss/python/langchain/agents>

2 21 22 35 45 54 Customize Deep Agents - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/customization>

3 11 27 38 Going to production - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/going-to-production>

4 59 Deep Agents overview - Docs by LangChain

https://docs.langchain.com/oss/javascript/deepagents/overview?utm_source=chatgpt.com

5 25 52 deepagents/libs/deepagents/pyproject.toml at main · langchain-ai/deepagents · GitHub

<https://github.com/langchain-ai/deepagents/blob/main/libs/deepagents/pyproject.toml>

6 39 Permissions - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/permissions>

7 Custom middleware - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/middleware/custom?utm_source=chatgpt.com

9 Tools - Docs by LangChain

<https://docs.langchain.com/oss/python/langchain/tools>

10 20 Backends - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/backends>

12 55 Memory - Docs by LangChain

https://docs.langchain.com/oss/python/deepagents/memory?utm_source=chatgpt.com

13 Deep Agents overview - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/overview>

14 Subagents - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/subagents>

15 Trace LangChain applications (Python and JS/TS) - Docs by LangChain

<https://docs.langchain.com/langsmith/trace-with-langchain>

16 18 26 57 deepagents/libs/deepagents/deepagents/graph.py at main · langchain-ai/deepagents · GitHub

<https://github.com/langchain-ai/deepagents/blob/main/libs/deepagents/deepagents/graph.py>

17 Overview - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/middleware/overview?utm_source=chatgpt.com

19 deepagents/libs/deepagents/deepagents/__init__.py at main · langchain-ai/deepagents · GitHub

https://github.com/langchain-ai/deepagents/blob/main/libs/deepagents/deepagents/__init__.py

23 60 Streaming - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/streaming?utm_source=chatgpt.com

24 Event streaming - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/event-streaming?utm_source=chatgpt.com

28 62 Interrupts - Docs by LangChain

https://docs.langchain.com/oss/python/langgraph/interrupts?utm_source=chatgpt.com

29 36 37 Models - Docs by LangChain

<https://docs.langchain.com/oss/python/deepagents/models>

30 56 Streaming - Docs by LangChain

https://docs.langchain.com/oss/python/deepagents/streaming?utm_source=chatgpt.com

31 Streaming - Docs by LangChain

https://docs.langchain.com/oss/python/langgraph/streaming?utm_source=chatgpt.com

32 58 langchain-ai/deepagentsjs: The batteries included agent ...

https://github.com/langchain-ai/deepagentsjs?utm_source=chatgpt.com

33 Customize Deep Agents - Docs by LangChain

<https://docs.langchain.com/oss/javascript/deepagents/customization>

34 Backends - Docs by LangChain

https://docs.langchain.com/oss/javascript/deepagents/backends?utm_source=chatgpt.com

40 Sandboxes - Docs by LangChain

https://docs.langchain.com/oss/javascript/deepagents/sandboxes?utm_source=chatgpt.com

41 Evaluation concepts - Docs by LangChain

https://docs.langchain.com/langsmith/evaluation-concepts?utm_source=chatgpt.com

42 Test - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/test?utm_source=chatgpt.com

43 How to run evaluations with pytest

https://docs.langchain.com/langsmith/pytest?utm_source=chatgpt.com

44 How to read experiment results locally

https://docs.langchain.com/langsmith/read-local-experiment-results?utm_source=chatgpt.com

46 **Models | OpenAI API**

https://developers.openai.com/api/docs/models?utm_source=chatgpt.com

47 **GPT-5.6 Sol Model | OpenAI API**

https://developers.openai.com/api/docs/models/gpt-5.6-sol?utm_source=chatgpt.com

48 **Context engineering in Deep Agents**

https://docs.langchain.com/oss/python/deepagents/context-engineering?utm_source=chatgpt.com

49 **Async subagents - Docs by LangChain**

<https://docs.langchain.com/oss/python/deepagents/async-subagents>

53 **deepagents**

https://www.npmjs.com/package/deepagents?utm_source=chatgpt.com

61 **Graph API overview - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/graph-api?utm_source=chatgpt.com